

HLD R2 — Restricción de acceso a recursos privilegiados

Qué es este documento. Diseño de alto nivel (HLD) del requerimiento R2: *restringir el acceso a recursos privilegiados solo a usuarios autorizados*. Es **diseño conceptual**, no de software: el detalle de implementación va en el LLSD. Vale **10 puntos** del Ex1. Es además el requerimiento cuyo **código se entrega** y cuya **demo en vivo** se hace durante el parcial: despliegue de un slice predefinido. Los números entre corchetes son los pesos de la rúbrica. **Sobre la topología:** el diseño describe una red de campus genérica. El laboratorio VNRT es solo el banco donde se valida y se demuestra (§9); el diseño no depende de su topología concreta. **Convención:** (*medido*) = experimental y reproducible. (*objetivo*) = compromiso sin verificar aún. (*analítico*) = cálculo sobre un modelo.

1. Planteamiento del problema

R1 ya estableció *quién* es cada usuario. R2 responde la pregunta siguiente: **¿a qué tiene derecho a llegar?**

La amenaza que hay que contener es específica y conviene nombrarla bien: **no es un ataque desde internet, es el acceso indebido desde dentro de la intranet**. Un alumno autenticado y legítimo que intenta alcanzar el servidor de notas es tráfico interno, entre dos puntos del campus, que **nunca pasa por el firewall perimetral**. Ese es el hueco que R2 cierra.

2. Inventario de recursos y criterio de clasificación [20]

Criterio de clasificación

Un recurso se clasifica por el **daño que causa su compromiso**, no por dónde está alojado ni por quién lo administra:

Clase	Criterio	Política por defecto
Crítico	Su alteración o filtración compromete la integridad académica o datos personales	Denegar , salvo rol explícitamente autorizado
Intermedio	Su caída interrumpe la operación docente, pero no compromete integridad	Permitir a roles autenticados, registrando el acceso
General	Servicio de consumo masivo, sin datos sensibles	Permitir a todo host autenticado

Inventario de campus

Propuesta de inventario y de esquema de direccionamiento. Los recursos críticos se agrupan en una **subred de servidores**, lo que permite expresar la política en pocas reglas agregadas (§6).

Recurso	Direccionamiento propuesto	Puertos	Clase	Roles con acceso
Servidor de notas / actas	10.20.0.10/32	443	Crítico	Docente, Superusuario
Repositorio de exámenes	10.20.0.20/32	22, 443	Crítico	Superusuario
Sistema de matrícula	10.20.0.30/32	443	Crítico	Docente, Superusuario
Servidores de laboratorio	10.20.1.0/24	22, 443	Intermedio	Todo rol autenticado
Portal académico, DNS, DHCP	red de servicios	53, 80, 443	General	Todos
Northbound API de gestión	10.20.2.0/24	8080	Crítico	Administrador de red, Superusuario

Cobertura: todo recurso crítico tiene política explícita. No hay recurso sin clasificar, que es lo que la rúbrica evalúa como precisión del inventario.

Principio de ubicación

Los recursos críticos se concentran **detrás de un punto de agregación** (switch de distribución o núcleo), no dispersos entre switches de acceso. Consecuencia de diseño: **todo acceso cliente → recurso crítico atraviesa un punto donde la política se puede aplicar de forma completa y auditable**, sin depender de que todos los switches de acceso tengan la misma capacidad.

3. Mecanismo de restricción y su justificación [20]

Se adopta RBAC aplicado como políticas OpenFlow en la tabla de política, leyendo el rol desde *metadata* que escribe R1.

Alternativa	Qué ofrece	Por qué se descarta
ACL estáticas por IP	Sin controlador; soportado por cualquier switch	La regla se ata a la IP del cliente: con DHCP la IP cambia y la política se rompe. No expresa <i>rol</i> ni reacciona a eventos
Firewall perimetral	Maduro, inspección profunda	Es un punto único en el borde: no ve el tráfico intra-campus , que es justo la amenaza. Desviarlo todo al firewall crea cuello de botella y <i>tromboning</i>
ABAC (por atributos)	Máxima expresividad (hora, ubicación, postura)	Complejidad de evaluación y prueba desproporcionada para cuatro roles. Evolución del Ex2
RBAC sobre OpenFlow ← adoptado	El rol viaja en <i>metadata</i> ; la decisión se aplica distribuida, en cada switch, a velocidad de línea	—

Razones decisivas:

1. **Aplica donde ocurre la amenaza.** La política vive en el switch, así que el intento se bloquea en el primer salto, sin atravesar la red. Un firewall perimetral simplemente no vería ese tráfico.
2. **Desacopla identidad de dirección.** La regla coincide con el **rol** en *metadata*, no con la IP del cliente. Un docente conserva su permiso aunque cambie de IP, y **el número de reglas no crece con la población** (§6).
3. **Es reactivo por diseño.** Consume *ataque_detectado* y *sesion_revocada* y extiende el bloqueo sin esperar un nuevo *Packet-In*.
4. **Está validado que el mecanismo existe:** el soporte de *metadata*, *goto_table*, *meters* y grupos se comprobó experimentalmente (*medido*).

4. Reglas de acceso: claridad y coherencia [20]

Modelo declarativo de política

La política se expresa de forma declarativa y se compila a reglas. Este documento es el *slice* que se despliega:

```
slice: campus-recursos-privilegiados
version: 1
politica_por_defecto: denegar

recursos:
- nombre: servidor-notas
  destino: 10.20.0.10/32
  puertos: [443]
  roles_permitidos: [docente, superusuario]

- nombre: repo-examenes
  destino: 10.20.0.20/32
  puertos: [22, 443]
  roles_permitidos: [superusuario]
```

```
- nombre: sistema-matricula
destino: 10.20.0.30/32
puertos: [443]
roles_permitidos: [docente, superusuario]
```

```
registro:
  denegaciones: true
  timeout_regla_denegacion_s: 10
```

Traducción a reglas OpenFlow

Cada entrada del slice se compila a reglas de la tabla de política, dentro de los rangos del contrato: **30000-39999 permisos, 40000-49999 denegaciones.**

#	Prioridad	Coincidencia	Acción	Origen
1	39000	rol=docente, dst=10.20.0.10, tcp=443	continuar a reenvío	permiso notas/docente
2	39000	rol=superusuario, dst=10.20.0.10, tcp=443	continuar a reenvío	permiso notas/superusuario
3	39000	rol=superusuario, dst=10.20.0.20, tcp=22	continuar a reenvío	permiso repo/ssh
4	39000	rol=superusuario, dst=10.20.0.20, tcp=443	continuar a reenvío	permiso repo/https
5	41000	dst=10.20.0.10	descartar + registrar	denegación explícita notas
6	41000	dst=10.20.0.20	descartar + registrar	denegación explícita repo
7	30000	(sin coincidencia de recurso crítico)	continuar a reenvío	tráfico general

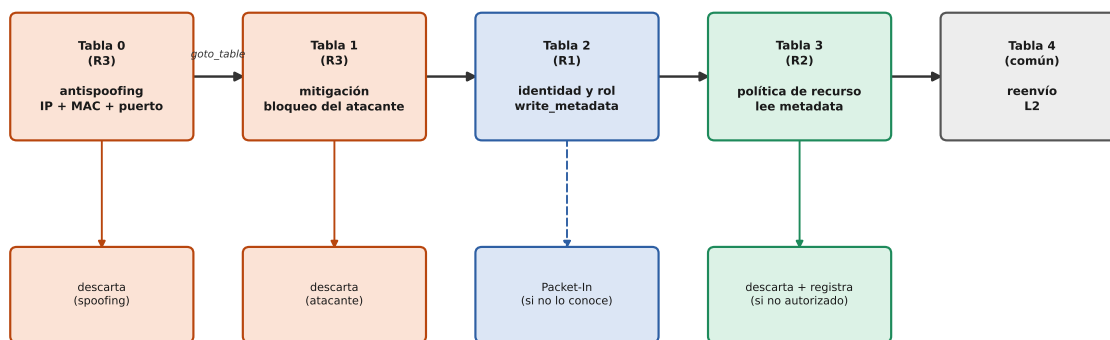
Coherencia garantizada por construcción. Los permisos se evalúan antes que las denegaciones; lo que no coincide con ningún permiso cae en la denegación explícita del recurso. **No hay solapes ni vacíos:** todo paquete dirigido a un recurso crítico termina en una regla de permiso o en una de denegación. El resto pasa por la regla 7. Esta propiedad es demostrable leyendo la tabla, y es lo que se muestra en la demo (§9).

Ciclo de vida de una denegación

La regla de denegación se instala con **tiempo de vida corto** (10 s) en lugar de ser permanente. El propósito es doble: **registrar el intento y no acumular entradas** en la tabla. El efecto secundario es deseado: un atacante insistente vuelve a generar **Packet-In** al expirar la regla, lo que alimenta a R3 con la señal de un patrón de acceso repetido. La restricción y la detección se retroalimentan.

5. Coherencia con la arquitectura general [20]

R2 ocupa la **tabla de política** y los rangos **30000-49999** del contrato (`../contratos/tablas-openflow.md`). Lee el rol desde `metadata` —escrito por R1— y **nunca lo modifica**.



Pipeline de tablas OpenFlow y su relación con R1, R2 y R3.

Por qué el orden importa para R2:

- R2 **nunca** ve tráfico de un host ya marcado como atacante: la tabla 1 lo cortó

antes. No se gasta evaluación de política en tráfico ya condenado.

- R2 **siempre** encuentra el rol en **metadata**: si R1 no lo hubiera escrito, el

paquete no habría llegado a la tabla de política. La dependencia es explícita y unidireccional, que es lo que permite desarrollar los módulos en paralelo.

R2 consume	De	Efecto
host_autenticado	R1	Conoce la asociación host ↔ rol
sesion_revocada	R1	Retira permisos sin esperar un nuevo Packet-In
ataque_detectado	R3	Propaga el bloqueo al resto de switches

R2 emite	Hacia	Carga útil
acceso_denegado	R3	MAC origen, IP destino, recurso

6. Estimación de entradas de TCAM por switch [10]

Advertencia conceptual obligatoria. Un switch por software **no tiene TCAM**: usa tablas en memoria más una caché en el kernel. Todo lo de esta sección es un **cálculo analítico sobre un modelo de switch de hardware**, y se presenta separado de cualquier medición hecha en software. Mezclarlas sería un error conceptual.

Fórmula

$E_{R2} =$ Suma sobre cada recurso crítico r de:
 $(puertos_r \times roles_permitidos_r)$
 $+ N_recursos$ (una denegación explícita por recurso)
 $+ 1$ (denegación por defecto, agregada)

Lo esencial en una frase: **E_{R2} depende de recursos × roles, no del número de usuarios.**

Desglose para el inventario propuesto

Origen de la regla	Entradas	Justificación
Permisos por rol y recurso	6	notas: $443 \times \{docente, super\} = 2$ · repo: $\{22, 443\} \times \{super\} = 2$ · matrícula: $443 \times \{docente, super\} = 2$
Denegaciones explícitas	3	una por recurso crítico
Denegación por defecto	1	entrada agregada, no una por host

Origen de la regla	Entradas	Justificación
Total R2	10	

Proyección a escala

Escenario	Recursos críticos	Puertos/recurso	Roles permitidos	E_R2
Inventario propuesto	3	~1.3	~1.7	10
Campus moderado	20	2	2	101
Campus grande	50	3	3	501

El argumento clave: las reglas se agregan por recurso y por rol, **no por host**. Un diseño que instalara una regla por usuario daría, en un campus de 10 000 usuarios, del orden de **10 000 entradas solo para R2** — inviable en hardware real, donde una TCAM típica ronda las 2 000-8 000 entradas. Con agregación por rol, 20 recursos protegidos caben en **~101 entradas**: dos órdenes de magnitud menos, y **constante frente al crecimiento de la matrícula**.

Presupuesto total por switch (analítico)

Módulo	Entradas	Crece con
R1 (identidad)	1 por host activo en ese switch	usuarios conectados a ese switch (acotado por su número de puertos)
R2 (política)	10 propuesto · ~101 campus	recursos × roles
R3 (mitigación)	≤ 50 (<i>objetivo</i>), con temporizadores	atacantes activos; se autolimpia
Reenvío L2	1 por MAC aprendida	hosts activos

Contraste con lo medido en software (no es TCAM)

En el banco de validación, el switch por software instaló **50 000 reglas sin rechazar ninguna**, con un costo **plano de ~140 µs por regla** y sin degradación entre 5 000 y 50 000 (*medido*). Es decir: **en el prototipo no hay problema de capacidad de tabla, ni de lejos**. Esa holgura **no sustituye** el cálculo analítico: un switch de hardware sí tiene un tope duro que el software no tiene. Por eso las dos cifras van separadas y con etiquetas distintas.

7. Eficiencia y carga sobre el controlador SDN [10]

Régimen permanente

R2 genera **Packet-In solo** en el primer contacto de un host con un recurso crítico, o al reintentar tras expirar una denegación. En régimen permanente, el tráfico autorizado se conmuta en el switch sin tocar el controlador.

Estimación: 200 docentes acceden al servidor de notas una vez por hora de clase → **≈ 0.06 Packet-In/s**. Frente al techo operativo medido de ~4 000 eventos/s, la carga de R2 en régimen permanente es **despreciable (analítico sobre datos medidos)**.

Peor caso: intentos no autorizados repetidos

Es el caso que sí acota el diseño. Con denegaciones de 10 s de vida, un host insistente genera como máximo **1 Packet-In cada 10 s por cada par (host, recurso)**: mientras la regla vive, el tráfico se descarta en el switch sin subir. Un atacante que rotara entre 100 recursos generaría ~10 Packet-In/s, tres órdenes de magnitud por debajo del techo.

Capacidad medida del controlador

Carga	Procesado	CPU	p95 latencia
100 /s	100 /s	4 %	1.45 ms
1 000 /s	1 000 /s	32-34 %	~1.5 ms
4 000 /s	4 000 /s	53-67 %	~1.6 ms
saturación	~11 000 /s (techo)	~110 %	> 4 s

(medido)

Conclusión: el presupuesto de control de R2 es holgadísimo. El riesgo real no es la carga legítima de R2, sino un **flood deliberado** que empuje al controlador a saturación, donde el p95 salta a segundos. La defensa correcta **no es del plano de control**: es aplicar *meters* —confirmados soportados— para limitar la tasa en el plano de datos antes de que el controlador se ahogue. Esto conecta directamente con la escalera de mitigación de R3.

8. Métricas comprometidas

Métrica	Objetivo	Estado	Cómo se mide
Precisión en la restricción	≥ 95 % de accesos no autorizados bloqueados	Pendiente de medir	tests/integration/, banco de ataques
Cobertura del inventario	100 % de recursos críticos con política explícita	Completa en el inventario propuesto (<i>diseño</i>)	Revisión del slice contra §2
Sesiones concurrentes soportadas	—	~4 000/s operativo · ~11 000/s techo (<i>medido</i>)	Rampa de carga
CPU del controlador	≤ 60 %	4 % a 100 eventos/s (<i>medido</i>)	Instrumentación del proceso
Entradas TCAM	≤ 50	10 para el inventario propuesto (<i>analítico</i>)	§6

Declaración honesta: la precisión de restricción (≥ 95 %) es el compromiso central de R2 y **todavía no está medida**. Depende del banco de ataques, aún no ejecutado. No se presenta ningún número estimado en su lugar: está diseñado, el procedimiento de medición está definido, y la medición es el siguiente paso.

9. Demo del parcial: qué se muestra en vivo

Guion del despliegue del slice, con el resultado observable en cada paso:

Paso	Acción	Resultado esperado
1	Desplegar el slice vía Northbound API	Reglas 1-7 instaladas en la tabla de política, verificable listando la tabla
2	Usuario con rol docente → servidor de notas	Permitido (regla 1)
3	Usuario con rol alumno → servidor de notas	Denegado (regla 5); el intento queda registrado
4	Usuario con rol alumno → repositorio de exámenes	Denegado (regla 6)
5	Listar la tabla de flujos	Ninguna regla fuera del rango 30000-49999
6	Repetir el paso 3 antes de 10 s	Descartado en el switch, sin nuevo Packet-In

El paso 5 demuestra que el contrato de tablas se cumple —es decir, que los tres módulos pueden convivir sin pisarse— y el paso 6 demuestra el control de carga sobre el controlador.

10. Caso de uso asociado

CU-02, especificado en el AVZ03, se mantiene íntegro. Este HLD desarrolla su mecanismo: el inventario, la clasificación, el modelo de política, la traducción a reglas y el presupuesto de tabla.